

Automatic Data Repair: Are We Ready to Deploy?

Wei Ni[*‡1], Xiaoye Miao[*★2], Xiangyu Zhao[‡3], Yangyang Wu[#4], Shuwei Liang[*#5], Jianwei Yin[*#†6]
[*]Center for Data Science, Zhejiang University, Hangzhou, China
[★]The State Key Lab of Brain-Machine Intelligence, Zhejiang University, Hangzhou, China
[‡]School of Data Science, City University of Hong Kong, Hong Kong, China
[#]School of Software Technology, Zhejiang University, Ningbo, China
[†]College of Computer Science, Zhejiang University, Hangzhou, China
{[1]wei.ni, [2]miaoxy, [4]zjuwuyy, [5]lsw5221}@zju.edu.cn [3]xianzhao@cityu.edu.hk [6]zjuyjw@cs.zju.edu.cn

ABSTRACT

Data quality is paramount in today's data-driven world, especially in the era of generative AI. Dirty data with errors and inconsistencies usually leads to flawed insights, unreliable decision-making, and biased or low-quality outputs from generative models. The study of repairing erroneous data has gained significant importance. Existing data repair algorithms differ in information utilization, problem settings, and are tested in limited scenarios. In this paper, we compare and summarize these algorithms with a driven information-based taxonomy. We systematically conduct a comprehensive evaluation of 12 mainstream data repair algorithms on 12 datasets under the settings of various data error rates, error types, and 4 downstream analysis tasks, assessing their error reduction performance with a *novel but practical* metric. We develop an effective and unified repair optimization strategy that substantially benefits the state of the arts. We conclude that, it is always worthy of data repair. The clean data does not determine the upper bound of data analysis performance. We provide valuable guidelines, challenges, and promising directions in the data repair domain. We anticipate this paper enabling researchers and users to well understand and deploy data repair algorithms in practice.

PVLDB Reference Format:
Wei Ni, Xiaoye Miao, Xiangyu Zhao, Yangyang Wu, Shuwei Liang, Jianwei Yin. Automatic Data Repair: Are We Ready to Deploy? . PVLDB, 17(10): 2617 - 2630, 2024. doi:10.14778/3675034.3675051

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/WelkinNi/Automatic-Data-Repair.

1 INTRODUCTION

The global application landscape relies heavily on data, while ubiquitously erroneous information involved in data often compromises the data analysis performance, leading to a huge economic cost [17, 43, 49, 84]. Dirty data may trigger the spread of fraud information, inaccurate decision-making, and even legal repercussions [42, 60, 62, 79, 86]. Among the strategies for efficiently mitigating erroneous information and ensuring data quality, *automatic data cleaning* [39, 86] has become a *pivotal* solution, especially in the age of generative artificial intelligence (GAI), where the large volume of high-quality training data for powerful GAI models like ChatGPT [67] and Midjourney [11] currently often resort to *resource-intensive manual* data cleaning [87].

The promising automatic data cleaning task is generally conducted in two consecutive stages: *error detection* and *error repair*. The error detection stage is to identify all wrong data values or rule violation sets, while the error repair stage is to correct these wrong values into latent right ones [2, 56, 57]. Existing error detection studies [38, 57, 65, 71] have achieved obvious advancements, with an average F1 score exceeding 0.85 on real datasets [57, 71]. In contrast, the error repair is more complex and challenging. Various techniques for error repair have been employed, such as machine learning [56, 72, 85], transfer learning [36, 56], boosting algorithm [47] and few-shot learning [56]. However, within the advanced detection results, the *final* data repair performance of current methods still falls short of the expected ideal, with an average F1 score below 0.7 in real scenarios [32, 56, 72]. Thus, it is crucial and urgent to analyze and explore how to deploy *end-to-end* data cleaning approaches with effective *data repair* algorithms in real-life scenarios.

Table 1 compares and summarizes existing surveys of end-to-end data cleaning study. They either analyze the data repair process without experiments [17, 43, 44, 48], or verify the impact of data repair on downstream tasks while overlooking the inclusion of certain important algorithms [1, 51]. The limitations and analysis perspectives come from the following *three* aspects.

L1: Metric shortfalls in evaluation. Fewer data errors typically lead to higher data quality and reduced biases in subsequent analyses, which constitute the *main goals* of the data cleaning task. Prior data cleaning surveys [1, 43, 44, 48, 51] lack an evaluation of *error reduction* in final data repair results, as indicated in Table 1. They often evaluate the data repair results with metrics such as *precision* and *recall*, which are based on the proportion of correctly repaired cells, disregarding the absolute quantity changes. Thus, such metrics result in a *biased* evaluation. For example, suppose there is a dataset that initially encompasses 10 erroneous cells. One data cleaning approach repairs 100 cells, of which 80 are correct (including the initial 10 error cells). Despite 70 of 80 values being identical to their originals, they are still flagged as erroneous values, and replaced by a value within calculation, thereby should be considered as a repaired cell. It means that, the dataset currently contains

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 17, No. 10 ISSN 2150-8097.
doi:10.14778/3675034.3675051

Table 1: Comparison of previous surveys and ours.

Survey study	Exp. evaluation	Error reduc. evaluation	Error rate		Error type	
			DRE*	DME*	DRE*	DME*
Ilyas et al. [43, 44]	✗	✗	✗	✗	✗	✗
Krishnan et al. [48]	✗	✗	✗	✗	✗	✗
Chu et al. [17]	✗	✗	✗	✗	✗	✗
Li et al. [51]	✓	✗	✗	✗	✗	✓
Abdelaal et al. [1]	✓	✗	✓	✗	✗	✗
Ours	✓	✓	✓	✓	✓	✓

* DRE and DME refer to data repair and downstream model evaluation.

20 erroneous cells, indicating a *doubles* amount of erroneous cells after the repair process. While the precision and recall of this repair both reach as high as 0.8 and 1.0, respectively. The correct repair of more initially right data increases the precision, according to the mediant inequality, but it cannot benefit the data quality. As a result, a higher precision/recall value does not represent a larger degree of error reduction. Furthermore, for distance-based metrics like Jaccard distance or mean squared error (MSE), they only measure the distance between the repaired and clean data, while *ignoring* the distance between the original dirty data and clean data. In a situation where the initial distance between dirty and clean data is 0.1, which then increases to 0.15 after repair. Simply stating the distance as 0.15 fails to directly indicate an increase in data errors. It cannot convey the actual *improvement degree* in data quality, either. Incorporating error reduction degree into evaluations will align repair algorithms more closely with improving data quality, and offer a direct perspective on data quality changes.

L2: Insufficient scenarios exploration in evaluation. The effectiveness of data repair algorithms differs in different scenarios. As depicted in Table 1, existing surveys lack the study of data repair algorithms' capacity under various *error types*, in either the data repair performance evaluation or the assessments of its impact on downstream tasks [1, 17, 44, 48, 51]. Meanwhile, the *error rate* analysis is absent in downstream impact evaluation, even though it is included in repair performance assessments. It results in an inadequate understanding of the proper deployment specific to distinct error types and error rates. It is vital and practical to comprehensively evaluate data repair algorithms with a diverse range of error types and rates in both repair and downstream tasks.

L3: Lacking algorithm deployment guidelines. Blindly deploying data repair algorithms may potentially hurt data quality and task performance. Although existing surveys discuss data repair algorithm performance [17, 43, 44, 48], *three* critical questions regarding algorithm deployment remain unanswered: (i) Is data repair consistently beneficial irrespective of the error rate? (ii) Based on the observation that, the suboptimal performance of data repair primarily results from incorrect repairs, how to develop a practical strategy to mitigate incorrect repairs based on error reduction evaluation? (iii) How to select the proper repair method in different scenarios based on task objectives? The data repair process plays a crucial role in the cleaning task, and addressing these issues will provide practical guidance to deploy data cleaning algorithms.

Therefore, in this survey, we conduct an exhaustive comparative analysis and experimental evaluation of 12 state-of-the-art data repair algorithms. Our main contributions are described below.

- We systematically evaluate *twelve* mainstream and state-of-the-art data repair algorithms based on a driven information-based taxonomy of data repair algorithms, including *cstr-driven* data repair algorithms, *data-driven* ones, and *hybrid-driven* ones. We also state the properly applied scenarios and graphically represent the workflow within each category.
- We define a novel and effective *metric* named *Error Drop Rate* to evaluate the error reduction condition of final data repair results. Surprisingly, in the majority of cases, most data repair algorithms tend to introduce more errors rather than eliminate erroneous information. We propose a *universal* optimization strategy that leverages error detection techniques to prevent the alteration of original right values into wrong ones.
- Extensive experiments on *twelve* real datasets under various error rates/types and data scales demonstrate both the performance of *twelve* data repair algorithms and the impact on *four* common downstream tasks. Particularly, except for five algorithms with publicly available codes, we implement *seven* additional data repair algorithms for comprehensive comparisons. We investigate the effect of *semantic* and *syntactic* data errors (that simulate complex practical scenarios) on the algorithm performance.
- We conclude several interesting findings: *i)* Repair algorithms often *increase* errors. Contrary to expectations, most existing data repair algorithms tend to raise the error rate in the data rather than eliminate it; *ii)* Using the clean data is *not* always the best. Downstream analysis models trained on purely clean data may perform worse than models trained using dirty data. *iii)* Data repair could provide universal benefits for downstream tasks. With proper algorithm selection, data repair boosts downstream tasks in the vast majority of cases; *iv) Semantic* errors pose a more stubborn challenge. Semantic errors within the data are harder to eliminate and have a more negative impact on downstream data analysis than *syntactic* ones, highlighting the need for greater consideration in real-world applications.

In the rest of this paper, Section 2 presents the preliminaries, while the new taxonomy and algorithm analysis are covered in Section 3. Comprehensive evaluations are offered in Section 4. Section 5 discusses current challenges and future directions, followed by related work in Section 6. Finally, Section 7 concludes the study.

2 PRELIMINARIES

In this section, we first introduce the problem definition. We then describe the constraints widely used in automatic repair algorithms.

Data cleaning mainly serves two primary goals: eliminating errors and ensuring data consistency, with the latter being a subgoal of the former. In this survey, we aim to evaluate *error reduction* performance over the final data repair results, as stated below.

Definition 1. **Problem Statement.** Given an instance I of relation R characterized by a set of attributes $Attrs = \{A_1, A_2, \cdots, A_n\}$. Each $t \in I$ comprises a set of cells represented by $Cells[t] = A_i[t]$ corresponding to a distinct attribute in $Attrs$. For each cell c, its unknown true value is denoted by v_c^*, its initial observed value is v_c, and its estimated true value is $\hat{v}_c$. A data repair algorithm aims to make $\hat{v}_c$ equal to v_c^* for all cells c.

This error reduction evaluation defined in *Definition* 1 encompasses correcting all erroneous cells within a dataset. It shares the same definition of *holistic repair*, as studied in most recently proposed data repair algorithms [56, 57, 71, 72].

Expert experience and field knowledge of the data serve as a critical guideline in the data repair process. These guidelines manifest as rules and constraints (cstrs). Rules provide explicit instructions for modifying erroneous values, making data repair a deterministic process [26, 34]. However, real-world scenarios often lack external sources of authoritative information, such as master data or expert insights. This absence poses a significant challenge in employing rules in the data repair process [34]. In contrast, constraints are designed to capture the relationships between specific attributes or values. Constraints have a *wider* range of applications, since professional external information may not always be accessible.

Numerous automatic data repair algorithms are built based on constraints like *functional dependency* (FD) [69] and *denial constraint* (DC) [70], as stated below.

Definition 2. **Functional Dependency.** A functional dependency is a statement of the form $X \rightarrow A$ where $X \subseteq Attrs$ and $A \in Attrs$, denoting that the values of attribute set X uniquely determine the values of attribute A across all tuples in I. X and A are the left-hand and right-hand attributes, respectively.

Definition 3. **Denial Constraint.** A denial constraint φ is a statement of the form $\forall t_\alpha, t_\beta \in I : \neg (p_1 \wedge \cdots \wedge p_m)$ where p_i is in the following form: $t_\alpha.A_x \ \phi \ t_\beta.A_y$ or $t_\alpha.A_x \ \phi \ c$, $A_x, A_y \in Attrs$, c is a constant, and ϕ is a built-in operator, i.e., $\neq$, $=$, $\leq$, $<$, $\geq$, $>$.

Example 1. A sample instance from real-world tax data [4] is presented in Table 2, along with two real FDs and two DCs:

$F_1 : City \rightarrow State$,

$DC_1 : \forall t_1, t_2 \in I : \neg(t_1.City = t_2.City \wedge t_1.State \neq t_2.State)$,

$DC_2 : \forall t_1, t_2 \in I : \neg(t_1.City = t_2.City \wedge t_1.Salary \geq t_2.Salary \wedge t_1.Rate \leq t_2.Rate)$.

The FD F_1 means if the values of $City$ are the same, then the values of $State$ should also be the same. Meanwhile, DC_1 asserts that no two tuples in R can have identical $City$ values paired with differing $State$ values. DC_2 indicates that within a given $City$, a higher $Salary$ should not correspond to a lower $Rate$ of tax. In this case, F_1 and DC_1 are equivalent, expressing the same meaning. The erroneous cells violating these constraints are colored in gray.

An FD in the above form $X \rightarrow A$ can be equally expressed in the following DC format: $\forall t_\alpha, t_\beta \in I$, $\neg(t_\alpha.X_0 = t_\beta.X_0 \wedge \cdots \wedge t_\alpha.X_k = t_\beta.X_k \wedge t_\alpha.A \neq t_\beta.A)$, $X_0, \cdots, X_k \in X$. While DC has greater expressive ability than FD. For instance, the monotonic relationship between values in DC_2 cannot be captured by FD, underscoring the expressivity gap between them. Considering the expressive ability and broader application spectrum, we mainly focus on DC-based data repair algorithms when evaluating the repair solutions adopting constraints.

Based on constraints, to achieve data consistency, previous work [8, 18] also offers other repair definitions, which are significant in algorithm understanding and deployment process:

Definition 4. **Tolerant Repair.** Given a set of constraints Σ over a relation schema R, along with a set of modified constraints Σ', and two instances I and I' of R. (Σ', I') is a tolerant repair of I if I' adheres to the modified constraint set Σ'.

Definition 5. **Consistency Repair.** Given a set of constraints Σ over a relation R, and two instances I and I' of R. I' is a consistency repair of I if I' adheres to the constraint set Σ.

Table 2: Relation of tax.

Tuple	FirstName	LastName	Gender	City	State	Salary	Rate
t_1	Weiming	Posthoff	Female	Vera	Okla.	45,000	6.25
t_2	Weiming	Zongtian	Male	Okemos	Mich.	10,000	3.9
t_3	Shivraj	Alpin	Female	Eastham	Mass.	90,000	5.3
t_4	Yurdaer	Thackray	Female	Eastham	LA.	30,000	5.2

3 DATA REPAIR ALGORITHMS

In this section, we elaborate 12 mainstream data repair algorithms based on a new *information-driven* taxonomy. We present a general and effective strategy to optimize these data repair algorithms.

3.1 Algorithm Taxonomy

Automatic data repair, which typically relies on data and/or constraint information, aims to enhance data quality or improve the performance of downstream tasks.

To offer insights into how they utilize constraints and data according to the driven information, we categorize existing data repair algorithms into three groups, i.e., *constraint(cstr)-driven*, *data-driven*, and *hybrid-driven* methods. The *cstr-driven* methods repair data based on constraints, typically employing *equivalent classes* (which are sets of cells that should satisfy the same constraint conditions). The *data-driven* algorithms only utilize data distribution information (without constraints) to repair data. For *hybrid-driven* algorithms, except for constraints, they also leverage data information not covered in constraints to facilitate data repair.

This taxonomy is useful for providing clear guidelines on using constraints and data, and offering targeted methods for different error types (cstr-driven for semantic, data-driven for syntactic, and hybrid for general errors). It also aligns with varying needs in deployment. Cstr-driven methods for regulated scenarios like healthcare and finance, and adaptable data-driven or hybrid methods for fields like machine learning tasks.

Moreover, there are *three* primary repair objectives within these algorithms, namely *data consistency*, *holistic repair*, and *model performance boost*. Data consistency encompasses *consistency repair*, which is applied to data with strict constraints like, geographic data where city names rely on states, and *tolerant repair*, used when constraints are either outdated or incorrect. *Holistic repair* focuses on maximizing error removal, crucial in fields like cybersecurity or healthcare. Lastly, the aim of *model performance boost* is to refine training data to enhance outcomes of data-driven tasks, such as image classification. Regarding these objectives, cstr-driven algorithms universally aim to achieve consistency repair. Data-driven approaches are for holistic repair and enhancing model performance. While the hybrid-driven methods are oriented toward tolerant repair or holistic repair.

Guided by the taxonomy, we select methods based on the utilization of constraints and data. In *cstr-driven* methods, we examine widely cited Holistic [18] and Nadeef [22] for mainstream graph-based and statistic-based data modeling with DCs. For *data-driven* ones, methods predominantly encompass statistical boosting and machine learning (ML)-based techniques, with BoostClean [47] as the only example for the former, Baran [56] and Scare [85] pioneered for ML applications in data repair. *Hybrid-driven* tools like Relative [8] and Unified [16] are representative in heuristic-based methods, while HoloClean [72] merges constraints with data



Can Foundation Models Wrangle Your Data?

Avanika Narayan
Stanford University
avanika@cs.stanford.edu

Ines Chami
Numbers Station
ines.chami@numbersstation.ai

Laurel Orr
Stanford University
lorr1@cs.stanford.edu

Christopher Ré
Stanford University
chrismre@cs.stanford.edu

ABSTRACT

Foundation Models (FMs) are models trained on large corpora of data that, at very large scale, can generalize to new tasks without any task-specific finetuning. As these models continue to grow in size, innovations continue to push the boundaries of what these models can do on language and image tasks. This paper aims to understand an underexplored area of FMs: classical data tasks like cleaning and integration. As a proof-of-concept, we cast five data cleaning and integration tasks as prompting tasks and evaluate the performance of FMs on these tasks. We find that large FMs generalize and achieve SoTA performance on data cleaning and integration tasks, even though they are not trained for these data tasks. We identify specific research challenges and opportunities that these models present, including challenges with private and domain specific data, and opportunities to make data management systems more accessible to non-experts. We make our code and experiments publicly available at: https://github.com/HazyResearch/fm_data_tasks.

PVLDB Reference Format:
Avanika Narayan, Ines Chami, Laurel Orr, and Christopher Ré. Can Foundation Models Wrangle Your Data?. PVLDB, 16(4): 738 - 746, 2022.
doi:10.14778/3574245.3574258

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/HazyResearch/fm_data_tasks.

1 INTRODUCTION

Foundation Models (FMs) [19] are models trained on broad data that can be adapted to a wide range of downstream tasks. These models have achieved substantial gains across many semantically challenging tasks such as question answering [20], knowledge base construction [79], and information retrieval [39]. As they have scaled to hundreds of billions of parameters (e.g. GPT-3 [20], PaLM [22]), large FMs have demonstrated surprising emergent behaviors and good zero-shot generalization to new tasks (i.e. no task-specific finetuning) on domains vastly different from the data they were pretrained on [22]. These large FMs are often autoregressive language models (e.g. GPT-3 and PaLM) that are trained to predict the next word in large text corpora and can be adapted to new tasks given a simple natural language description of the task (see Figure 1). These breakthrough capabilities have led to a race for building bigger and better models, and innovations continue to push the boundaries of what large FMs can do on a variety of hard *language tasks*.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 4 ISSN 2150-8097.
doi:10.14778/3574245.3574258

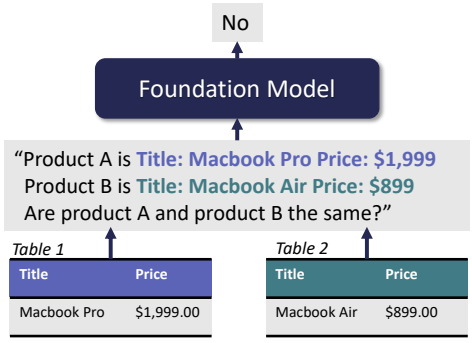


Figure 1: A large FM can address an entity matching task using prompting. Rows are serialized into text and passed to the FM with the question "Are products A and B the same?". The FM then generates a string "Yes" or "No" as the answer.

A natural question that arises is whether these advances can benefit hard classical *data tasks* (e.g. data cleaning and integration). While it is clear that FMs benefit text-intensive tasks, it is not clear whether these models can be applied to data tasks over structured data. The symbols commonly found in structured data (e.g. dates, numbers, alphanumeric codes) are less frequent in natural language text so it is unclear that FMs possess the ability to reason over them. Moreover, since FMs are trained to predict the next word, it is non-obvious that they can work out-of-the-box on complex data tasks. This paper explores the aforementioned question and introduces a new research vision for leveraging FMs for data management, focusing on data cleaning and integration tasks—two keys steps in data-driven enterprise pipelines.

Recently, a large body of research has applied machine learning (ML) [49] and deep learning (DL) [57, 74] methods—namely pretrained language models (PLMs) like BERT [32]—to semantically-complex data tasks. However, these approaches still require a significant amount of engineering effort as they rely on:

- **Task-specific architectures**: Data cleaning and integration encapsulate *many different tasks* such as entity matching [77], schema matching [92], and error detection [41]. Existing approaches, whether they are rule-, ML- or DL-based, vary greatly from one task to the other, often with complex, task-specific architectures. For instance, adapting BERT to data tasks requires architectural changes and finetuning the entire model for each task. This leads to siloed and hard-to-maintain systems.
- **Hard-coded knowledge**: Data tasks often rely on *domain knowledge* (e.g. understanding the relationship between a city and its zip code for data cleaning constraints) and commonsense reasoning. These are usually hard-coded with human-engineered rules or external knowledge bases [24, 84]. Consequently, systems can be brittle and fail to generalize to a diverse set of domains.

- **Labeled data**: ML- and DL-based solutions require copious amounts of hand-labeled data [9]. For instance, PLMs that have achieved state-of-the-art (SoTA) results on data tasks (e.g. Ditto [38]) require a significant amount of task-specific labeled data and fine-tuning to achieve good performance. Labeling data for each task is engineering intensive and adds to the difficulty of maintaining data cleaning and integration systems.

Excitingly, FMs display several useful properties that make them an appealing choice compared to traditional approaches:

- **Task-agnostic architecture**: As a result of their natural language interface, FMs can be applied to a wide-range of tasks. For instance, Figure 1 shows how an entity matching task—which requires identifying whether two table entries refer to the same entity—can be cast as a prompting task. This unifying interface eliminates the need for siloed architectures, in contrast to existing learned approaches where architectures need to be carefully crafted for each task (e.g. task-specific classification layer).
- **Encoded knowledge**: Because FMs are trained on large, generic corpora of data, they contain knowledge about an extensive set of common entities, and thus do not rely on human-engineered rules to acquire knowledge [82].
- **Limited to no labeled data**: FMs can be applied to a breadth of tasks with little to no labeled data (e.g. few-shot and zero-shot). When a FM needs to be fine-tuned, it typically needs dramatically less labeled data to achieve competitive results [46].

Our goal is to better understand if large FMs can be applied to data integration and cleaning tasks. We study the behavior of GPT-3—an early and promising FM. While GPT-3 is already a high quality model, we expect the significant investment in FMs from both academia and industry to lead to more performant and scalable FMs over time. Like many other communities, the data management community stands to benefit from these trends. As such, we aim to understand the advantages and limitations of FMs on data tasks, by focusing on three key questions.

How well do large FMs transfer to data tasks? To answer this, we cast several data tasks as natural language generation tasks (Section 3) and explore whether a single FM can generalize well to these tasks. In Section 4.2, we quantify the zero- and few-shot performance of FMs on five enterprise data tasks: entity matching, error detection, schema matching, data transformation, and data imputation. We find that the largest GPT-3 variant (175B parameters) outperforms SoTA ML-and DL-based approaches on these tasks with few examples. This is particularly surprising since prior approaches are fully-finetuned on task-specific labeled data for these tasks, while GPT-3-175B is simply pretrained to generate text.

What are the caveats in applying FMs to data tasks? In Section 4.3, we unpack the few-shot "prompt tuning" process—serializing tabular data to text, casting data tasks as text generation tasks and constructing demonstrative task examples—for applying FMs to data tasks. We quantify the effects of prompt formatting variations on performance and the differences between manually and randomly selecting task examples. We find that FMs are brittle to differences in prompt formatting and that performance improves when prompts are manually selected versus randomly selected.

What opportunities do FMs present for data tasks and what are the relevant research challenges? Finally, in Section 5, we discuss the potential challenges and related research questions with using FMs in data management pipelines. We discuss the forthcoming shift in how ML systems are built, challenges around updating FM knowledge, and opportunities and considerations pertaining to private, temporal and local data.

We hope that our preliminary exploration will encourage the data management community to explore the effectiveness of FMs for other data tasks and develop techniques to overcome the shortcomings of FMs in this setting.

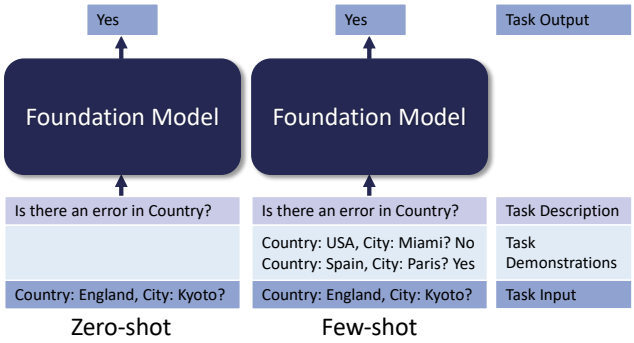


Figure 2: Different ways to use FMs with "in-context" learning [20] on an error detection task. For zero-shot (left), the prompt is the task description and the example to complete. For few-shot (right), the prompt adds task demonstrations.

2 BACKGROUND

We first give some background on the different data tasks considered in this paper and then provide a brief review of FMs.

2.1 Problem Setup

We focus on entity matching (EM), error detection (ED), and data imputation (DI) and describe the setup for these tasks. We denote D, a structured dataset with n entries, such that each entry is represented by a collection of m attribute value pairs: for entry $e_i \in D$ we have $e_i = \{e_{i,j}\}_{1 \le j \le m}$ where for attribute j, $e_{i,j} = \{\texttt{attr}_j, \texttt{val}_j\}$.

Entity Matching The goal of EM is to match entities (real-world objects like people, places and things) across different datasets. Formally, given two structured datasets (D, D') and pairs of entries $e, e' \in D \times D'$, the goal is to predict whether these entries represent the same entity or not. This problem is usually solved as a classification problem, and real-world EM systems are often preceded by blocking heuristics which are used to remove obvious non-matches.

EM has been extensively studied over the past decade (see [77] for a survey) and methods broadly fall into three categories: rule-based, crowd-based [36, 96] and ML/DL-based [49, 74]. Recently, methods relying on PLMs [57] have become SoTA for this task.

Error Detection ED is an important step in data cleaning pipelines. Given an entry e, the goal is to detect attributes j where $\texttt{val}_j$ has an error. The task is framed as a classification problem where the goal is to predict if $\texttt{val}_j$ is correct for a given e.

ED has been studied extensively in both academic and industrial settings [7]. There are a number of successful commercial

offerings including Trifacta [6] and Tamr [5]. Traditionally, ED systems have been heavily reliant on rule-based algorithms which enforce data constraints through functional dependencies or knowledge bases [23, 24, 26]. Additionally, there are statistical-based approaches such as pattern enforcement [24, 45], outlier detection [28], and record deduplication [89] algorithms. Recent efforts have developed SoTA ML models for ED [41].

Data Imputation DI is a critical step for repairing dirty data sources. Given an entry e with missing attribute values $\{\texttt{attr}_j, \texttt{NULL}\}$, the goal of DI is to infer the missing values. The full range of plausible values for the missing value is not known apriori.

Prior works in DI falls into three categories—clustering and/or statistical-based [70], generative model-based [35], ML/DL-based [17, 72], and tabular data pretraining-based approaches [30]—and struggle when needing to impute values not seen in the training set [72].

2.2 Background on Foundation Models

We now give an overview of language FMs, starting from very early, smaller FMs (i.e. PLMs) and moving to large-scale FMs, the latter of which is the focus of this paper.

Pretrained Language Models Pretrained Language Models (PLMs) are neural networks pretrained on large corpora of publicly available text (e.g. web pages). The first breed of PLMs—ELMo [78], BERT [33], RoBERTA [64]—learned the semantics of natural language by predicting masked works during pretraining. These models have on the order of hundreds of millions of parameters. Traditionally, PLMs are adapted to downstream tasks through a task-specific prediction layer (e.g. classification layer) and a task-specific finetuning step wherein all model weights are updated.

Large Autoregressive Language Models In 2020, GPT-3 [20] marked a significant shift in the the ML community. It represented a new class of large-scale language models: autoregressive language models pretrained to predict the next word in a sequence. These models have *billions* of parameters and have been used for language generation tasks such as question answering and summarization. Since the release of GPT-3, bigger and better performing autoregressive language models have been developed [22].

Emergent Behaviors Interestingly, the biggest GPT-3 variant (175B parameters) has the capacity to solve natural language tasks with only a few examples (few-shot prompting), and, in some cases, just a task description (e.g. "Translate French to English"). Unlike traditional finetuning, no model parameters are updated to fit the task. Few-shot prompting has proven to be effective on tasks widely different from the FMs pretraining objective (e.g., code generation [100], Trivia QA [20, 59] and arithmetic [20]). This in-context learning behavior can be thought of as the FM "locating" an already-learned behavior [85]. Smaller models (<10B parameters) typically require some form of task-specific finetuning. We study ways to use smaller FMs on data tasks in the full report [75].

3 FOUNDATION MODELS FOR DATA TASKS

Our goal is to understand whether FMs can benefit data cleaning and integration tasks. The procedure for applying FMs to data tasks requires adapting structured data inputs to textual inputs (Section 3.1), casting data tasks as text generation tasks (Section 3.2) and, for few-shot prompting, constructing demonstrative task examples to help the FM learn new data tasks (Section 3.3).

3.1 Tabular Data Serialization

FMs take text as input and generate text as output. In order to apply these models to data tasks, we first need to convert structured tabular data inputs to text representations.

Given a structured dataset, we first convert an entry to text. Concretely, for entry e, we follow previous work [57] and serialize the attribute value and entry as follows:

$$\texttt{serialize}(e) \coloneqq \texttt{attr}_1 : \texttt{val}_1 \ldots \texttt{attr}_m : \texttt{val}_m$$

If the attribute value is NULL, it is serialized as the empty string. Based on the task and dataset, serialization only happens over a subset of attributes relevant to the task. FMs can be sensitive to the prompt format and the specific serialization used [105]. In Section 4.3 we study the sensitivity of FMs to sub-selection choices.

3.2 Data Tasks as Natural Language Tasks

Next, we need to convert data tasks to text generation tasks. We construct natural language descriptions of each task (i.e. prompts) that use the serialized representations from Section 3.1. The prompts are passed to the FM, whose generated output is the answer to the given task. For entity matching and error detection, the model generates a "Yes" or "No" response,[1] and for imputation it generates the missing value. We now enumerate the prompts for each task.

Entity matching: given two entries (e, e') the template is

```
Product A is serialize(e). Product B is serialize(e').
Are Product A and Product B the same?
```

Data imputation: given an entry e and attribute j to infer, we use

$$\texttt{attr}_1 : \texttt{val}_1 \ldots \texttt{attr}_j \ ?$$

Error detection: given an entry e and attribute j to classify as erroneous, we use

$$\texttt{Is there an error in attr}_j : \texttt{val}_j ?$$

These templates highlight the generality of this framework which could be extended beyond the tasks considered in this paper.

3.3 Task Demonstrations

Task demonstrations can be included in the prompt to help the model learn a new task (see Figure 2 for an error detection example). These demonstrations are used to show the model how the task should be completed (e.g. should it generate Yes/No or a missing value) as well as understand the finer-grained semantics of D. We explore two approaches for selecting task demonstration examples.

Random One approach is to sample random examples from a labeled dataset. However, this approach often causes high variance in FM performance [62]. Moreover, the ordering of examples in the

[1] Interestingly, the model is not constrained to produce a Yes/No answer on the output side, but we find that this happens most of the time. In the rare examples where the model does not predict a Yes/No answer, we use No as the default answer.

DataRinse: Semantic Transforms for Data preparation based on Code Mining

Ibrahim Abdelaziz
IBM Research
Yorktown Heights, New York
ibrahim.abdelaziz1@ibm.com

Julian Dolby
IBM Research
Yorktown Heights, New York
dolby@us.ibm.com

Udayan Khurana
IBM Research
Yorktown Heights, New York
ukhurana@us.ibm.com

Horst Samulowitz
IBM Research
Yorktown Heights, New York
samulowitz@us.ibm.com

Kavitha Srinivas
IBM Research
Yorktown Heights, New York
kavitha.srinivas@ibm.com

ABSTRACT

Data preparation is a crucial first step to any data analysis problem. This task is largely manual, performed by a person familiar with the data domain. *DataRinse* is a system designed to extract relevant transforms from large scale static analysis of repositories of code. Our motivation is that in any large enterprise, multiple personas such as data engineers and data scientists work on similar datasets. However, sharing or re-using that code is not obvious and difficult to execute. In this paper, we demonstrate DataRinse to handle data preparation, such that the system recommends code designed to help with the preparation of a column for data analysis more generally. We show that DataRinse does not simply shard expressions observed in code but also uses analysis to group expressions applied to the same field such that related transforms appear coherently to a user. It is a human-in-the-loop system where the users select relevant code snippets produced by DataRinse to apply on their dataset.

PVLDB Reference Format:
Ibrahim Abdelaziz, Julian Dolby, Udayan Khurana, Horst Samulowitz, and Kavitha Srinivas. DataRinse: Semantic Transforms for Data preparation based on Code Mining. PVLDB, 16(12): 4090 - 4093, 2023.
doi:10.14778/3611540.3611628

1 INTRODUCTION

Enterprises often have many data analysts performing modeling and exploration on the same or similar types of datasets. One of the most time consuming parts of any data analysis step is the preparation of data, such as encoding categorical values, handling ordinals, imputation of missing values, data aggregations, amongst others. This step often requires a significant amount of domain knowledge, and is largely carried out manually by a data engineer, data scientist, analyst or a domain expert. The knowledge, once applied, is often buried in code, and this code tends to be stored in enterprise repositories. Listing 1 shows an example of such a transform, where the data scientist is trying to bin `Age` so that it can be combined with the passenger class attribute to make a combined inference about the wealth and age of the passenger, which likely impacted survival on the Titanic. This snippet of code is adapted from a much larger script that performs multiple operations on the titanic dataset[1]. A key goal of DataRinse is to allow a data scientist re-use of code relevant to the dataset without them having to peruse all the code that manipulates the same dataset.

In DataRinse, we perform this by applying static code analysis on Python scripts. We adopt the path of static analysis compared to dynamic analysis because very often the datasets associated with code in repositories are not available readily making dynamic analysis impractical. It works by finding the transformations to fields of a dataframe, which are then extracted, and converted into an intermediate representation, and this representation is traversed to generate code to help the user transform their dataframe. A key goal of DataRinse is to behave as a recommendation system rather than automating data preparation in one shot. Code generation for data preparation is particularly challenging because: (a) the order in which specific transformations are performed is important; (b) code generation has to be sensitive to the 'groups' of transformations that constitute a single operation conceptually. We illustrate both points with a running example. Listing 1 shows a code snippet from an actual script for preparing the titanic dataset for exploratory analysis. Note that when code is generated, it needs to be aware that a number of transforms are being performed as a 'group', so all the contiguous `Age` transforms bin `Age` into buckets. Furthermore, the transformation of `Age * Pclass` is dependent on that binning operation being performed first; that is, there is a dependency between the first function and the second.

DataRinse performs both grouping of conceptually related operations into functions and encodes dependencies between several functions to offer users functions that can be easily incorporated into their own notebooks. We use control flow from static analysis to perform grouping, and we use control and data flow to understand dependencies between the functions that are generated in DataRinse. Furthermore, to make this consumable to the end user, we display a drop down box in the notebook, so that they can choose which field they want to view cleansing functions for. For each function that is extracted, we also show the URL of the

[1]https://github.com/davified/clean-code-ml/blob/master/notebooks/titanic-original.ipynb

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 12 ISSN 2150-8097.
doi:10.14778/3611540.3611628

script that was used to generate that function. Listing 4 shows the output of DataRinse of a code snippet for the Age column, where a single function captures all binning changes to Age, and a second separate function shows the creation of a new column using it. The user has a number of such functions per column that can be readily applied to their dataset if they deem it applicable, and the ordering of transformations is encapsulated in the code generated such that a call to transform Age is performed before the use of it in expressions such as `Age * Pclass`, as shown. All the generated functions are dynamically evaluated on the dataframe to ensure that any functions that are not compile-able are filtered out. Similarly, the generated functions that do not result in a change in the values of the dataframe are also eliminated.

We note that ChatBots such as ChatGPT[2] can appear to provide similar functionality for such tasks. There are however key differences between those systems and DataRinse. While ChatGPT generates some useful data operations, it has the following characteristics that are different from DataRinse: (a) It has non-determinism in its generation, it can sometimes find cleansing operations, but produces different results on each run; a well-known problem for ChatBots (see Listings 2, 3 for two different code snippets produced for the same dataset (b) It does not follow basic data operation rules such as being consistent across train and test, and it is not comprehensive - `Fare` is handled only in one snippet. (c) There is no attribution to the source that these are drawn from, (d) It is not focused - the snippets were drawn from a much larger script, where the system produced imports, read CSV files, etc. (e) It can only work off its training dataset - DataRinse can for instance run over any scripts in any repository at any desired frequency (e.g., on every commit or daily) requiring negligible computational resources; ChatBots have to be retrained on enterprise specific code.

2 SYSTEM OVERVIEW

Figure 1 describes the overall DataRinse system for data preparation. Given a new dataset, the system queries code repositories such as GitHub with the table's metadata, specifically the table and column names. One current limitation of DataRinse is that it will only select scripts that match column names and table names - this is a limitation we plan to address in future work through more flexible column-to-concept mappers [7]. The analysis framework is an open source project that we created[3] which performs inter-procedural static analysis to create control flow and data flow graphs for Python scripts [1]. The scope of the analysis is the entire notebook, so dataflow and control flow can be used to order the cleansing operations. We perform interprocedural analysis to trace both data and control flow, as illustrated in Figure 2. The chalky lines for `read_df` and `prep_df` denote function calls, and dataflow follows them, e.g `"titantic_train.csv"` flows to `fl_name`. After analysis, the code generation component for cleansing starts by isolating reads, writes and updates to pandas dataframe. It uses dataflow information to determine which expressions are relevant, e.g. dataflow connects the read of a pandas dataframe at the top of Figure 2 to the update of `dataset` at the bottom.

[2]https://openai.com/blog/chatgpt
[3]https://github.com/wala/graph4code/tree/master/semForms

```python
import pandas as pd

def read_df(fl_name):
  return pd.read_csv(fl_name)

def prep_df(combine):
    for dataset in combine:
        dataset.loc[ dataset['Age'] <= 16, 'Age'] = 0
        dataset.loc[(dataset['Age'] > 16) &
        ↪  (dataset['Age'] <= 32), 'Age'] = 1
        dataset.loc[(dataset['Age'] > 32) &
        ↪  (dataset['Age'] <= 48), 'Age'] = 2
        dataset.loc[(dataset['Age'] > 48) &
        ↪  (dataset['Age'] <= 64), 'Age'] = 3

def main():
    train_df = read_df("titanic_train.csv")
    test_df =  read_df("titanic_test.csv")
    combine = [train_df,test_df]
    prep_df(combine)
    for dataset in combine:
        dataset['Age*Class'] = dataset.Age *
        ↪  dataset.Pclass
```

Listing 1: Example data preparation code, handling the titanic dataset

```python
train_df = train_df.drop(['PassengerId', 'Name',
↪  'Ticket', 'Cabin'], axis=1)
train_df['Age'].fillna( train_df['Age'].median(),
↪  inplace=True)
train_df['Embarked'].fillna(
↪  train_df['Embarked'].mode()[0], inplace=True)
train_df['FamilySize'] = train_df['SibSp'] +
↪  train_df['Parch'] + 1
train_df['IsAlone'] = 1
train_df['IsAlone'].loc[train_df['FamilySize'] > 1]
↪  = 0
train_df['Sex'] = train_df['Sex'].map({'female': 0,
↪  'male': 1}).astype(int)
```

Listing 2: Code snippets from ChatGPT, when prompted to prepare the titanic dataset for analysis, first attempt

The code generation component assembles related code into multiple functions, organized by the fields that they change, as well as records any dependencies in the application of the functions by encoding calls to relevant functions. In Pandas, calls on a dataframe can sometimes mutate the dataframe (e.g. `drop()`), and sometimes not (e.g. `isnull()`). Because there is no way of knowing if the values of a dataframe are actually modified by a call, the filtering component dynamically evaluates the functions returned by the code generation module in the client notebook to filter out functions that do not transform the data frame. Extracted functions across

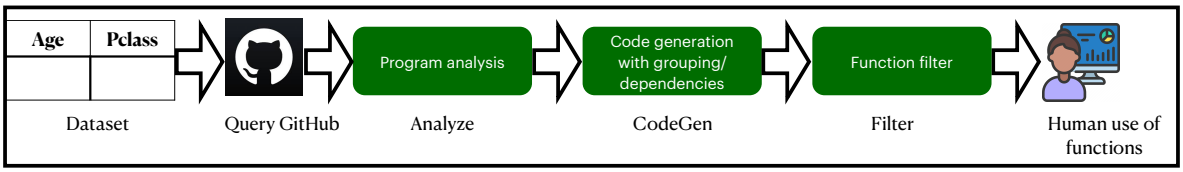


Figure 1: Overview of DataRinse

```
train_df.drop(['PassengerId', 'Name', 'Ticket',
  ↪  'Cabin'], axis=1, inplace=True)
test_df.drop(['Name', 'Ticket', 'Cabin'], axis=1,
  ↪  inplace=True)
train_df['Age'].fillna( train_df['Age'].median(),
  ↪  inplace=True)
test_df['Age'].fillna(test_df['Age'].median(),
  ↪  inplace=True)
test_df['Fare'].fillna( test_df['Fare'].median(),
  ↪  inplace=True)
```

Listing 3: Code snippets from ChatGPT, when prompted to prepare the titanic dataset for analysis, second attempt

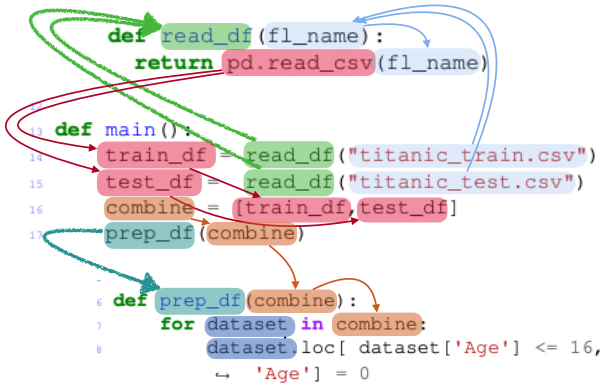


Figure 2: Analysis on listing 1

multiple scripts are then grouped by field, and each function for a given field encapsulates the extraction from a single script. The user can choose which functions they would like to look at, and explore what they would like to add to their own code through an interactive interface.

3 DEMONSTRATION

We show in Listing 4 an example of how DataRinse recommends data transforms is used for the example code shown in 1, which is a simplified version of the original code in[4], for the titanic dataset[5]. The UI is structured as a Jupyter notebook where the user loads

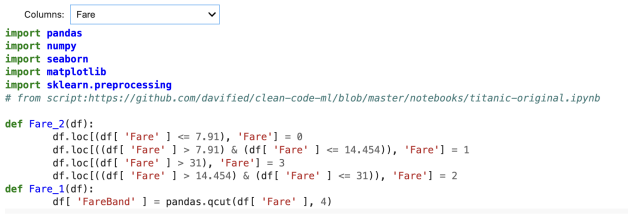


Figure 3: Display of expressions by columns

a dataset. The dataset name and columns are fed to GitHub to retrieve scripts that may be cleansing the same dataset. The scripts are analyzed, and code generation is performed for each field in that is manipulated in the scripts. Any dependencies in manipulations are captured by the code generation as well, as shown in Listing 4. A second example is shown in Listing 5 for the UCI heart disease prediction dataset[6]. Notice here that the order of operations in the original script[7] are maintained, which involves setting missing values in the resting ECG results to `normal` before using a `LabelEncoder` to encode all the values.

In general, there may be too many data manipulation operations across scripts. Since the generated code relies on an IR, obvious duplicates are eliminated at creation time across scripts. Scripts are also ordered by the ones that have the most number of functions across fields, because those are the most comprehensive. The code suggestions are organized by columns using a dropdown as shown in Figure 3. At the conference, we will allow the audience to select an available dataset, and explore the code segments provided by the system. We will also work with the audience to use a dataset of their choice.

4 EXPERIMENTS

To assess the generality of DataRinse as a tool, we applied it to 1,589 Python notebooks from GitHub that were converted into Python scripts for analysis. Table 1 shows some statistics of running DataRinse over these scripts. Approximately 11% (182) of the 1,589 scripts had failures because the code could not be parsed or was not actually Python code; so that left us with 1407 scripts. Analysis ran successfully on most of those scripts (1,406); but after removal of duplicate scripts, we had 1,142 scripts. Of the 1,142 scripts that were successfully analyzed, 55% yielded at least one cleansing function. The total number of CSV files mentioned in these scripts from which cleansing functions were extracted was 2,262, but we note that this is likely an overestimate since many scripts load train and test splits

[4]https://github.com/davified/clean-code-ml/blob/master/notebooks/titanic-original.ipynb
[5]https://www.kaggle.com/competitions/titanic/data

[6]https://www.kaggle.com/datasets/redwankarimsony/heart-disease-data
[7]https://www.kaggle.com/code/achintyak/decision-tree

Lingua Manga : A Generic Large Language Model Centric System for Data Curation

Zui Chen
IIIS, Tsinghua University
Beijing, China
chenzui19@mails.tsinghua.edu.cn

Lei Cao
U of Arizona/MIT
Tucson, Arizona
lcao@csail.mit.edu

Sam Madden
MIT
Cambridge, Massachusetts
madden@csail.mit.edu

ABSTRACT

Data curation is a wide-ranging area which contains many critical but time-consuming data processing tasks. However, the diversity of such tasks makes it challenging to develop a general-purpose data curation system. To address this issue, we present Lingua Manga , a user-friendly and versatile system that utilizes pre-trained large language models. Lingua Manga offers automatic optimization for achieving high performance and label efficiency while facilitating flexible and rapid development. Through three example applications with distinct objectives and users of varying levels of technical proficiency, we demonstrate that Lingua Manga can effectively assist both skilled programmers and low-code or even no-code users in addressing data curation challenges.

PVLDB Reference Format:
Zui Chen, Lei Cao, and Sam Madden. Lingua Manga : A Generic Large Language Model Centric System for Data Curation. PVLDB, 16(12): 4074 - 4077, 2023.
doi:10.14778/3611540.3611624

1 INTRODUCTION

In the era of big data, organizations are collecting vast amounts of data from various sources. However, the data is often messy, incomplete, or contains errors. To effectively perform data analytics or any other applications, it is often necessary to engage in a data curation process [8, 15], which includes tasks like data discovery, integration, and cleaning. Nonetheless, devising a data curation solution for a particular scenario can be a cumbersome and time-consuming process. It entails extensive communication of requirements, a collaboration between domain experts and programmers, multiple rounds of debugging and testing, and active maintenance to accommodate new use cases. This underscores the need for a *generic* system that enables users to efficiently and effectively address diverse data curation challenges and apply solutions to various applications.

However, building a system that addresses diverse data curation problems and applications is challenging. Data curation involves a range of distinct tasks, such as data discovery through table search, schema matching, entity resolution during data integration, and data imputation in data cleaning. Moreover, in real applications, various extra tasks are often involved in the data curation processes,

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 12 ISSN 2150-8097.
doi:10.14778/3611540.3611624

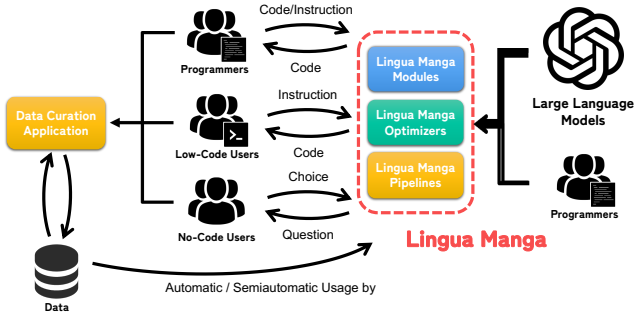


Figure 1: An illustration of the Lingua Manga System.

such as event extraction, name extraction, anomaly detection, data summarization, visualization, table semantic parsing, etc. Worst yet, applications in different domains have diverse requirements. Effectively resolving data curation problems often relies heavily on users' domain knowledge, making developing a *general-purpose* solution improbable. For example, entity resolution over *books* focuses on text understanding, while entity resolution over *images* should rather pay special attention to image processing. As a result, existing data curation systems typically focus on only a few tasks on specific data formats. For example, Data Tamer [8] solves schema matching and entity resolution problems; ZenCrowd [4] targets entity linking; while CrowdDB [7] addresses fuzzy SQL semantics.

Recent advances in Large Language Models (LLMs) have shed light on this highly challenging yet critical data curation area. Pre-trained LLMs such as GPT-3 [10], Codex [6], ChatGPT [17], and LLaMA [5] have not only shown the ability to understand human needs and instructions but also to generate code like a programmer. Such ability could be leveraged to solve specific data curation problems like data imputation or entity resolution [2].

However, the current use of LLMs and LLM plugins is still far from the ultimate solution. Repeatedly calling LLM services can be costly and may lead to data privacy issues. For instance, consider a large e-commerce organization with thousands of documents and tables containing billions of rows – it would be neither affordable nor secure to let an LLM access the entire data lake to complete the tasks. LLMs tend to be less effective without access to enterprise data due to the gap between the enterprise data and the public data LLMs trained on. Thus, there is an urgent need to harness the power of LLMs systematically, efficiently, and securely. Properly integrating LLMs and traditional data curation methods could be a promising path toward a general-purpose data curation system.

In this demonstration, we present Lingua Manga , an LLM-centric system that enables the swift development of effective yet efficient data curation solutions. It benefits programmers as well as low-code or even no-code users. Specifically, Lingua Manga shows the following key properties:

• **User-friendly**: Users can quickly develop a data curation solution by utilizing the templates, built-in modules, and, most importantly, by effectively interacting with LLMs through Lingua Manga as a middleware.

• **Flexible**: Users can effortlessly communicate with Lingua Manga through natural language (NL) to inject domain knowledge or instructions into the data curation solution such that it can meet the needs of the specific applications or domains.

• **Intelligent**: Lingua Manga automatically optimizes the solution to fix errors, improves performance, continuously learns from the data, etc.

• **Highly Performant**: Lingua Manga minimizes the frequency of calling the LLM service, which incurs heavy computation and monetary costs, thus resulting in an economical, efficient, and privacy-preserving solution.

• **Label Efficient**: with our system, users can develop a data curation solution with no or only a few labeled examples from the specific application while still achieving accuracy comparable to the SOTA ML-based methods trained with thousands of labels.

2 PRELIMINARIES: PROMPT-TUNING AND CODE GENERATION IN LLMS

Starting from GPT-3 [10], LLMs' size and computational cost have grown so much that users can no longer afford to fine-tune them for every downstream task. In the meantime, LLMs have also gained significant knowledge through training on large-scale corpora, leading to the emergence of a new paradigm known as prompt-tuning [9]. Using prompt-tuning, users can generalize LLMs to new tasks and data domains without modifying the LLM itself. This is achieved by incorporating prompts, such as descriptions, tags, and learnable components, into the input data.

Moreover, cutting-edge LLMs like ChatGPT [17] have demonstrated their effectiveness in interacting with humans using natural-language-based prompts [3]. These prompts are interpretable by humans. For instance, a straightforward prompt like "Please determine if the following entities are equivalent" can enable LLMs to carry out entity resolution. This capability is especially useful in Lingua Manga since it facilitates LLMs to interactively execute functions vaguely specified by users in natural language.

Furthermore, prompts like "Please write a code for entity resolution" trigger LLMs to generate custom software modules via code generation. GPT-3 [10], ChatGPT [17], Codex, and Copilot [6] are the mainstream LLMs that support code generation. However, existing methods that use these models to generate code have limitations [19]. They are typically inadequate for producing code with complex logic and building large-scale software. This calls for a new methodology to effectively utilize LLM Generated Code (LLMGC).

3 LINGUA MANGA

Lingua Manga comprises a Domain-Specific Language (DSL), a Compiler, an Optimizer, and a set of templates.

Lingua Manga is a **user-friendly** workflow system that enables users to quickly build data curation solutions by composing pipelines of logical operators. The system features a DSL to simplify the workflow-building process.

Lingua Manga is **automatic**. Like a relational database, it auto-compiles each logical operator into a physical, executable *module*.

Lingua Manga is **extensible**. Lingua Manga allows the programmers to implement their own physical modules, which can be seamlessly plugged into the system as long as they follow the standard interfaces of Lingua Manga .

Lingua Manga is **flexible**. To meet the specific need of different applications, the Lingua Manga optimizer interacts with users via LLMs to generate customized physical modules.

Lingua Manga is **intelligent**. The Lingua Manga optimizer uses LLMs to enhance the modules' capability.

In addition, to further facilitate users in building pipelines, Lingua Manga offers templates, essentially groups of pre-built pipelines. Rather than creating a pipeline from scratch, Lingua Manga allows users to start with a pre-defined, well-optimized pipeline that the target application can directly use.

The fundamental observation underlying this research is that while current LLMs may not yet be able to produce exceedingly intricate and large software systems, they are proficient at generating modular code on a smaller scale. Moreover, LLMs possess learning and logical reasoning ability that enable them to complete data curation tasks directly, indicating that LLMs could accelerate the process of developing a data curation solution by automating parts of it and, thus, reduce the workload of software developers.

Next, we discuss the modules and the Lingua Manga optimizer, which are the critical components of Lingua Manga .

3.1 Modules in Lingua Manga

A module is a function $f : X \rightarrow Y$ that takes X as input and returns Y. Modules are usually viewed as black boxes, and their behavior is determined by their expected output. In Lingua Manga , modules can be classified into four types:

• **Custom Module**: As a basic module, a custom module is implemented with manually written code. It can be created by users with programming skills or provided by Lingua Manga as a default built-in module.

• **LLM Module**: An LLM itself can be a module. An LLM module tends to be more powerful than a traditional module because of the common sense knowledge and logical inference abilities possessed by an LLM. However, an LLM module requires a good task description as input; and LLM outputs typically need proper validation, as textual responses generated by the LLMs could be diverse and unstable. For instance, numerical validation that ensures the LLM's output is within a reasonable value range can identify and correct potential errors in LLM output.

• **LLMGC Module**: An LLM can dynamically generate code to implement an LLMGC module, replacing the role of programmers. Lingua Manga allows LLMGC to call other modules in the system or use external tools such as a calculator or another pre-trained LLM. Although an LLMGC module might be less expressive than custom modules written by skilled programmers, it enables no-code or low-code users to implement or customize a module effortlessly.

• **Decorated Module**: As the most advanced module in Lingua Manga , a decorated module can comprise multiple basic modules and be enhanced by the optimizer that leverages the common sense knowledge encoded in the LLMs and their learning ability.

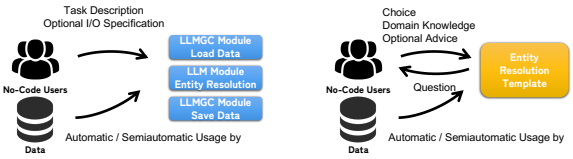
3.2 Lingua Manga Optimizer

The optimizer in Lingua Manga can improve the performance of physical modules within a pipeline by enhancing their efficiency and effectiveness, e.g., avoiding repeatedly calling the LLM service to minimize the execution costs and the potential data privacy leak. Unlike conventional database optimizers, the optimizer in Lingua Manga is modularized and can be selectively composed by users to serve specific applications best. The key modules in Lingua Manga include:

- **Validator**: It checks whether the target module behaves correctly on a few example test cases. It then uses the failed test cases to trigger the LLM to improve the target module and fix the errors. Specifically, the validator first calls an LLM to generate the suggestion by reading the code and the failure cases. Then, the code, failure cases, and the generated suggestion are sent to another LLM to generate a new version of the code. This validation cycle repeats until either all test cases are executed successfully, or a timeout ensues, leading to a re-generation of the LLMGC module until an additional timeout.
- **Simulator**: A simulator automatically generates a more efficient and equally effective alternative to a given module that already functions well. For example, a module that frequently calls LLM service can be expensive. Potentially, a supervised learning counterpart could replace it. This is because LLMs are versatile, while a data curation task typically only needs part of its functionality. It is thus possible to produce a module proficient in simulating the specific LLM functionality required by a task.

  Because each module is treated as a black-box function, an ML-based simulator can replicate the target module through supervised learning. The target module will function as intended during initialization, and a control logic will decide when the simulated version should take over, such as after achieving the desired accuracy or reaching a certain level of confidence.

  A simulator could improve accuracy as well. At first sight, this is counter-intuitive. Like the teacher-student model in machine learning, traditionally, the student model, which learns from a teacher model, is expected to be upper bounded by the teacher model on performance. However, studies have shown that self-training with filters [12, 16, 18, 20] can produce a superior student model because of better generalization. Moreover, the simulator will continuously monitor the real data flow in the original pipeline. It can thus constantly learn to adapt to the data distribution, potentially outperforming the original static module.
- **Connector**: Concerning efficiency and data privacy, it is crucial for applications to reduce the amount of data exposed to LLMs, while still ensuring the effectiveness. For instance, in an NL2SQL task, granting LLMs access to the entire table contents is neither secure nor cost-effective. However, relying solely on the table schema is not adequate to generate accurate predicates. To address this issue, a locally-running connector can be employed to manage the selective data upload to LLMs. A pre-defined connector for tabular data enables LLMs to execute SQL commands in local databases and obtain the resulting data while ensuring that the execution is limited to the queries specified by the user. In multi-modal data scenarios, appropriate pre-defined connectors are provided, such as connectors designed for handling extensive textual data.



(a) Custom pipeline. **(b) Builit-in template.**

Figure 2: An illustration of two possible Entity Resolution workflows using Lingua Manga .

4 DEMONSTRATION

We use three applications to demonstrate Lingua Manga . Fig. 5 shows its user interfaces with name extraction as an example task.

4.1 Entity Resolution: Effortless to the Novices

Consider the scenario where a technical novice wants to perform entity resolution on a dataset but lacks programming experience.

Lingua Manga effectively fulfills these requirements by providing a user-friendly solution. To begin, users can easily search for existing templates within the system. If none are available, they can create a simple pipeline that includes data loading, entity resolution, and data-saving operators, as depicted in Figure 2a. It is worth noting that the user does not have to write any code. Instead, they can simply describe the task to the LLMs using the suggested prompt templates and provide optional input and output specifications through examples. The Lingua Manga optimizer will then improve the performance automatically.

We evaluated the F1 scores of the solution produced by Lingua Manga on three entity resolution datasets: BeerAdvo-RateBeer, Fodors-Zagats, and iTunes-Amazon [1]. As shown in Table 1[1], it significantly outperforms the previous LLM-based method [2], while achieving performance comparable to supervised learning algorithms trained with hundreds or even thousands of labeled examples [13].

Table 1: Quantitative Experiment on Entity Resolution.

Dataset	Magellan [1]	Ditto [13]	FMs [2]	Lingua Manga
BeerAdvo-RateBeer	78.8	94.37	78.6	89.66
Fodors-Zagats	100.0	100.00	87.2	95.65
iTunes-Amazon	91.2	97.06	65.9	92.00

4.2 Name Extraction: Flexible for the Adepts

Consider a scenario where a domain expert, as a low-code user with basic programming skills, aims to get a high-accuracy and efficient name extraction solution, i.e., find all person names in a text passage. In this case, the domain expert understands that name extraction typically involves three steps: tokenization, noun-phrase extraction, and tagging. As a result, the domain expert can create a pipeline of three individual operators (ignoring data loading and saving for ease of presentation), as shown in Fig. 3.

The domain expert recognizes that tagging is a complex process and is often the performance bottleneck. Therefore, he or she chooses to utilize the LLM module within Lingua Manga along with an example-based validator. The other two operators are relatively simple and thus are realized as LLMGC modules – using LLM to generate the code. Suppose the resulting LLMGC modules, such as the noun-phrase extraction module, are not precise enough. The

[1]The numbers of the baselines are obtained from [2].